

Inventory Management System for a Retail Store

Java Programming, Case Study 37 Technology: Java (Swing GUI, Collections Framework)

1. Introduction

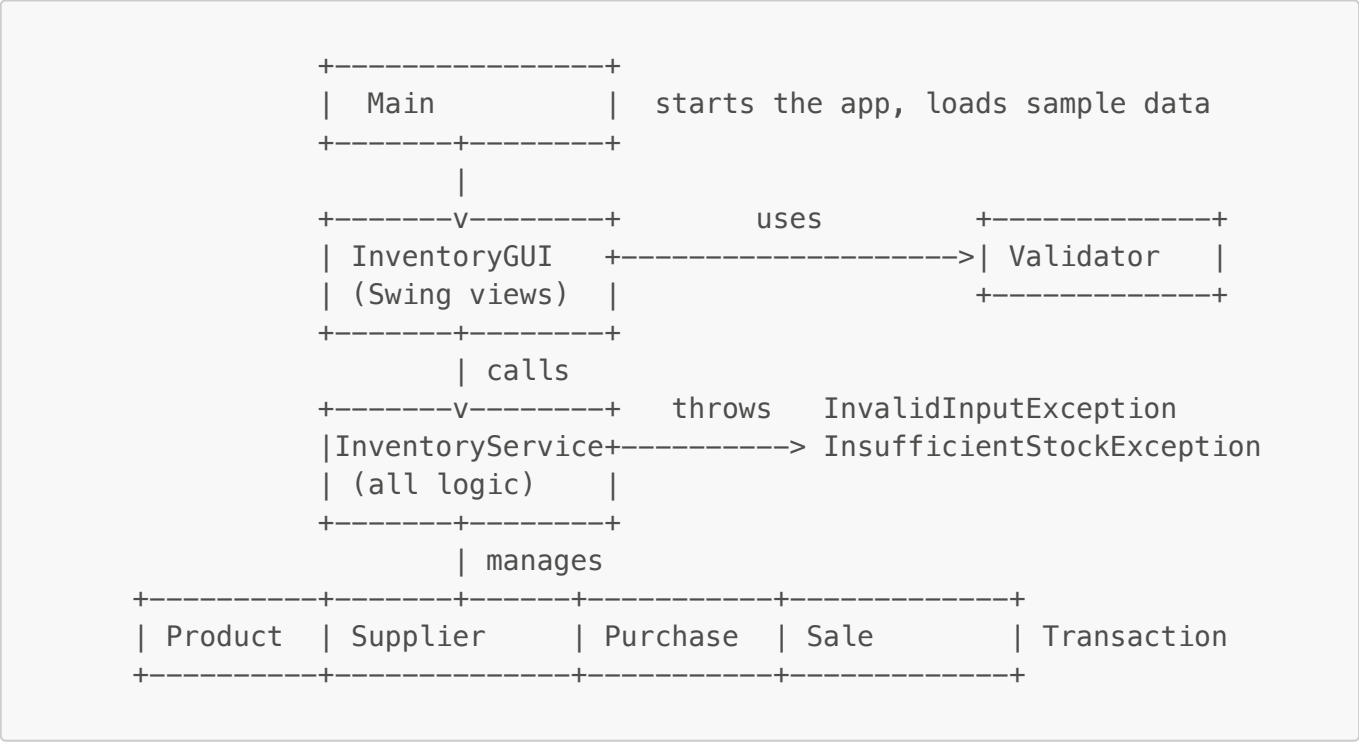
A retail store must keep track of what it sells, who supplies it, how much stock is on the shelf, and when to reorder. This project is a desktop application that manages products, suppliers, purchases, sales, billing and inventory history, and alerts the store manager when stock runs low.

2. Objectives

- Manage products and suppliers (create, read, update, delete).
- Track stock quantities automatically as purchases and sales happen.
- Record purchases and sales.
- Identify low-stock products and alert the manager.
- Search and sort products.
- Generate bills and inventory reports.

3. System Architecture

The project separates the user interface from the business logic.



InventoryGUI never changes data directly. Every action goes through **InventoryService**, which validates input and throws exceptions on invalid operations. The GUI catches them and shows dialogs.

4. Classes and Files

File	Purpose
<code>Main.java</code>	Entry point. Creates the service, loads sample data, opens the GUI.
<code>InventoryGUI.java</code>	Swing interface with six tabs.
<code>InventoryService.java</code>	All business logic: CRUD, stock, purchases, sales, search, sort, low-stock, billing, report.
<code>Product.java</code>	Product data: ID, name, category, price, quantity, supplier ID.
<code>Supplier.java</code>	Supplier data: ID, name, phone, email.
<code>Purchase.java</code>	A stock purchase from a supplier (auto-numbered, timestamped).
<code>Sale.java</code>	A sale line (auto-numbered, timestamped, price captured at time of sale).
<code>Transaction.java</code>	One entry in the inventory history (type, product, quantity change, note).
<code>Validator.java</code>	Central input validation rules.
<code>InvalidInputException.java</code>	Raised for bad input or an invalid transaction.
<code>InsufficientStockException.java</code>	Raised when a sale asks for more units than are in stock.

5. Java Concepts Used

Concept	Where and how it is used
Classes and objects	<code>Product</code> , <code>Supplier</code> , <code>Purchase</code> , <code>Sale</code> , <code>Transaction</code> model real-world entities.
Constructors	Each entity class initialises its fields through a constructor. <code>Purchase</code> and <code>Sale</code> also assign an auto-incremented ID using a static counter.
Arrays	<code>CATEGORIES []</code> and <code>THRESHOLDS []</code> hold the product categories and their reorder levels.
ArrayList	Stores suppliers, purchases and sales.
LinkedList	<code>history</code> stores inventory transactions. New entries are added at the front (<code>addFirst</code>) so the newest appears first.
HashMap	<code>Map<String, Product></code> maps Product ID to Product for O(1) lookup.
TreeMap	<code>byId()</code> gives products sorted by ID. <code>byPrice()</code> is a <code>TreeMap<Double, List<Product>></code> giving products sorted by price. Also used for the category totals in the report.
CRUD	Add, update and delete for products and suppliers.

Concept	Where and how it is used
Searching	By ID (direct HashMap lookup), by name and by category (case-insensitive partial match).
Sorting	By ID and price (TreeMap), by name and quantity (Comparator).
Swing	JFrame , JTabbedPane , JTable , DefaultTableModel , JComboBox , JOptionPane and a custom cell renderer.
Exception handling	Two custom checked exceptions, caught in the GUI and displayed as dialogs.
Validation	Validator class (see section 8).

6. Modules

1. Product Management. Add, update and delete products. Selecting a table row fills the form for editing. A product must reference an existing supplier. A new ID must not already exist.

2. Supplier Management. Add, update and delete suppliers. A supplier cannot be deleted while any product still uses it.

3. Stock Management. Stock changes automatically: purchases increase quantity, sales decrease it. Manual quantity edits are recorded as adjustments. Every change is logged in the history.

4. Purchase Management. Record a purchase by product ID, supplier ID, quantity and unit cost. Stock increases immediately and the purchase appears in the purchase table.

5. Sales Management. Sell by product ID and quantity. Stock decreases immediately. If the quantity requested is more than what is available, an [InsufficientStockException](#) is raised and nothing changes.

6. Product Search. Search by ID, name or category from the Products tab.

7. Product Sorting. Sort by ID, name, price or quantity.

8. Low Stock Alert. A product is low when its quantity is at or below the reorder level of its category:

Category	Reorder level
Grocery	20
Electronics	5
Clothing	10
Stationery	25
Household	8

Low stock is shown in four ways: red table rows, a warning popup at startup, a red status bar with the count, and a popup right after a sale takes an item below its level. The "Low Stock Alert" button lists all low products on demand.

9. Billing. Items sold form the current bill. "Generate Bill" prints an invoice with item lines, subtotal, 18% GST and grand total, then starts a fresh bill.

10. Inventory Report. Shows product and supplier counts, total stock value, total purchase cost, total sales revenue, stock value by category, the low-stock list and the last five transactions.

7. User Interface Guide

Tab	What you can do
Products	Manage products, search, sort, view low stock.
Suppliers	Manage suppliers.
Purchases	Record purchases and view purchase records.
Sales & Billing	Sell items, view the running bill, generate the invoice.
History	View all inventory transactions, newest first.
Report	View the inventory report. Refreshes after every change.

Search and sort results are temporary views. Any add, update, delete, purchase or sale refreshes every tab back to the full list.

8. Validation Rules

Field	Rule
Product ID	P followed by 3 to 6 digits (e.g. P001). Must be unique when adding.
Supplier ID	S followed by 3 to 6 digits (e.g. S001). Must be unique when adding.
Names	Cannot be empty.
Price and unit cost	A number greater than zero.
Quantity (product)	A whole number, zero or more.
Quantity (purchase, sale)	A whole number greater than zero.
Category	Must be one of the five defined categories.
Supplier on a product	Must be an existing supplier.
Phone	Exactly 10 digits.
Email	Standard email format.

9. Exception Handling

Exception	Raised when	Shown as
InvalidInputException	Validation fails, an ID is missing or duplicated, a supplier is still in use, or the bill is empty	Warning dialog

Exception	Raised when	Shown as
<code>InsufficientStockException</code>	A sale requests more than the available stock	Error dialog
Other <code>Exception</code>	Anything unexpected	Generic error dialog

Validation happens before any data changes, so a failed operation never leaves the inventory half-updated.

10. Data Structures at a Glance

Structure	Holds	Why this choice
<code>HashMap<String, Product></code>	All products	Fast lookup by ID ($O(1)$ average)
<code>TreeMap</code>	Sorted views by ID and by price	Keeps keys ordered automatically ($O(\log n)$)
<code>ArrayList<Supplier></code>	Suppliers	Small list, simple iteration
<code>ArrayList<Purchase></code> , <code>ArrayList<Sale></code>	Records	Append-only records
<code>LinkedList<Transaction></code>	History	Cheap insertion at the front for newest-first order
<code>String[]</code> and <code>int[]</code>	Categories and thresholds	Fixed, small lookup tables

11. Sample Data

The app starts with three suppliers and seven products.

ID	Product	Category	Price (Rs)	Qty	Supplier
P001	Basmati Rice 5kg	Grocery	480	45	S001
P002	Sunflower Oil 1L	Grocery	165	12	S001
P003	Wireless Mouse	Electronics	599	4	S002
P004	USB-C Cable	Electronics	249	30	S002
P005	Cotton T-Shirt	Clothing	399	25	S003
P006	Notebook A4	Stationery	60	18	S003
P007	Dish Wash Liquid	Household	95	40	S003

P002, P003 and P006 start below their reorder levels.

12. How to Compile and Run

Requirements: JDK 8 or later.

```
javac *.java
java Main
```

13. Test Cases

#	Action	Expected result
1	Launch the app	Low-stock popup lists P002, P003, P006; rows are red; status bar shows 3
2	Search Name rice	Only P001
3	Search Category elec	P003 and P004
4	Add product with ID X1	Invalid ID error
5	Add product with price -5	Invalid price error
6	Add product with supplier S999	Supplier not found
7	Add product with ID P001	Duplicate ID error
8	Add valid product P008	Appears in the table; NEW entry in history
9	Delete supplier S001	Refused, products still use it
10	Purchase 10 units of P003	Quantity becomes 14; no longer low
11	Sell 3 units of P004	Stock 27; bill shows subtotal, GST, total
12	Sell 100 units of P003	Insufficient stock error; stock unchanged
13	Sell 20 units of P005	Stock 5; low-stock warning popup
14	Generate bill with no items	"No items on the current bill"

14. Limitations and Future Enhancements

Current limitations

- Data is held in memory only and resets when the app closes.
- One user, with no login or roles.
- A sale deducts stock as soon as an item is added to the bill, and there is no undo for a sold line.
- Reorder levels are per category, not per product.

Possible enhancements

- Save and load data using files or a JDBC database (MySQL or SQLite).
- Login with manager and cashier roles.
- Export bills and reports to PDF or CSV.

- Per-product reorder levels and automatic purchase-order suggestions.
- Sales charts and date-range reports.

15. Conclusion

The system delivers a realistic retail inventory application. It combines Java Collections (ArrayList, LinkedList, HashMap, TreeMap and arrays), CRUD operations, searching, sorting, a Swing GUI, input validation and custom exception handling in a structure that keeps the interface separate from the business logic.